Funciones de grupo

SQL · Tercera parte | Resumir, agrupar y filtrar resultados

> **Ejemplos adaptados:** se usan tablas pequeñas para seguir los cálculos. Los resultados fueron calculados sobre los datos incluidos; no son copias de las capturas del apunte. Sin ORDER BY, no se garantiza el orden de las filas.

Las funciones de grupo, o de agregación, resumen muchas filas en **un solo valor**. Sin GROUP BY, resumen el conjunto completo; con GROUP BY, devuelven un valor por grupo.

1. Funciones que tenés que conocer

Función	Qué devuelve
AVG(columna)	El promedio de los valores no nulos.
SUM(columna)	La suma de los valores no nulos.
COUNT(*)	La cantidad de filas, tengan o no valores NULL.
COUNT(columna)	La cantidad de valores no nulos de esa columna.
COUNT(DISTINCT columna)	La cantidad de valores diferentes, sin contar NULL.
MAX(columna) / MIN(columna)	El mayor / menor valor no nulo.

2. COUNT · Filas, valores y valores distintos

Usaremos esta tabla Alumno en los ejemplos:

Leg	Nom	Ape	Cuota
69	Juan	Martínez	500
109	Juan	Pérez	600
138	Pedro	García	700
160	NULL	Gómez	NULL

```
SELECT COUNT(*) AS Filas,
       COUNT(Nom) AS Nombres,
       COUNT(DISTINCT Nom) AS NombresDistintos
FROM Alumno;
```

Filas	Nombres	NombresDistintos
4	3	2

Hay **4 filas**, pero solo **3 nombres no nulos**. Juan está repetido: los nombres distintos son Juan y Pedro, por eso el tercer resultado es **2**.

3. SUM y AVG · Los NULL no son ceros

```
SELECT SUM(Cuota) AS Suma, AVG(Cuota) AS Promedio
FROM Alumno;
```

Suma	Promedio
1800	600.00

El promedio es (500 + 600 + 700) / **3** = 600. La fila con cuota NULL no entra ni en la suma ni en el divisor.

> **Sin valores:** COUNT devuelve 0; SUM, AVG, MAX y MIN devuelven NULL si no hay valores no nulos para calcular. Si necesitás decimales, la columna o la expresión promediada debe tener un tipo numérico adecuado; AVG sobre enteros puede truncar en SQL Server.

Fuente: Universidad de Palermo, 4G0016_APU_Funciones_grupo_v1-0.pdf, pp. 1-10. Resumen reformulado y ejemplos adaptados.

Resumir exámenes

MAX, MIN y varios cálculos en una misma consulta

Tabla de referencia · Examen

Estos ocho exámenes se utilizan en todos los ejemplos siguientes. Nota admite decimales y NULL; Nroexamen identifica cada examen.

Nroexamen	Leg	Fecha	Mat	Nota
1	69	2016-02-15	1001	8
2	69	2016-02-15	1002	6
3	69	2016-07-15	1001	4
4	109	2016-02-15	1001	2
5	109	2016-07-15	1001	8
6	138	2016-02-15	1002	4
7	138	2017-02-12	1001	NULL
8	160	2017-02-12	1001	10

4. Mejor nota, peor nota y diferencia de un año

```
SELECT MAX(Nota) AS Mejor, MIN(Nota) AS Peor,
       MAX(Nota) - MIN(Nota) AS Diferencia
FROM Examen
WHERE Fecha >= '2016-01-01'
  AND Fecha < '2017-01-01';
```

Mejor	Peor	Diferencia
8	2	6

Primero WHERE conserva los **seis exámenes de 2016**. Sobre sus notas, MAX obtiene 8 y MIN obtiene 2; la diferencia es 6. No se necesita GROUP BY porque se calcula un resumen de todas las filas filtradas.

> **Fechas:** el intervalo desde el inicio de 2016 hasta antes de 2017 incluye todo el año, incluso si Fecha contiene horas. En el ejemplo 36 del original, la captura termina el 01/12/2016; ese límite no incluye el año completo.

5. Cantidad, máximo, mínimo y promedio de un alumno

```
SELECT COUNT(*) AS Cantidad, MAX(Nota) AS Mejor,
       MIN(Nota) AS Peor, AVG(Nota) AS Promedio
FROM Examen
WHERE Leg = 69;
```

Cantidad	Mejor	Peor	Promedio
3	8	4	6.00

Para el legajo 69 quedan las notas **8, 6 y 4**: rindió 3 exámenes y su promedio es 6. Podés combinar funciones y asignarles nombres con AS.

> **Contar exámenes no siempre es contar notas:** el legajo 138 tiene 2 exámenes, pero una sola nota registrada. COUNT(*) devuelve 2; COUNT(Nota), 1. Si Nroexamen nunca es NULL, COUNT(Nroexamen) equivale a COUNT(*).

GROUP BY

Una fila de resultado por cada grupo formado

6. Agrupar por alumno · Mejor nota de cada uno

```
SELECT Leg, MAX(Nota) AS MejorNota
FROM Examen
GROUP BY Leg
ORDER BY Leg;
```

Leg	MejorNota
69	8
109	8
138	4
160	10

GROUP BY Leg reúne todos los exámenes de cada legajo y MAX calcula el máximo **dentro de cada grupo**. La nota NULL del legajo 138 se ignora; su mejor nota es 4.

7. JOIN + GROUP BY · Agregar nombre y apellido

Para mostrar nombres, hay que juntar Examen con Alumno. Las columnas del SELECT que no están agregadas deben figurar en el GROUP BY, siguiendo el criterio del apunte.

```
SELECT E.Leg, A.Nom, A.Ape, AVG(E.Nota) AS Promedio
FROM Examen E INNER JOIN Alumno A ON E.Leg = A.Leg
WHERE E.Fecha >= '2016-01-01'
  AND E.Fecha < '2017-01-01'
GROUP BY E.Leg, A.Nom, A.Ape
ORDER BY E.Leg;
```

Leg	Nom	Ape	Promedio
69	Juan	Martínez	6.00
109	Juan	Pérez	5.00
138	Pedro	García	4.00

Dos errores frecuentes: mostrar A.Nom y A.Ape sin agregarlos al GROUP BY; o poner AVG(E.Nota) dentro del GROUP BY. Se agrupa por las columnas, no por la función que resume el grupo.

8. Agrupar por varias columnas

Para contar exámenes por fecha y materia, cada combinación **(Fecha, Mat)** forma un grupo distinto.

```
SELECT Fecha, Mat, COUNT(*) AS Cantidad
FROM Examen
GROUP BY Fecha, Mat
ORDER BY Fecha, Mat;
```

Fecha	Mat	Cantidad
2016-02-15	1001	2
2016-02-15	1002	2
2016-07-15	1001	2
2017-02-12	1001	2

En cada fecha hay dos exámenes por materia en este ejemplo. GROUP BY no garantiza un orden: para ordenar, se agrega ORDER BY.

HAVING y repaso

WHERE filtra filas; HAVING filtra grupos

9. Alumnos que rindieron más de dos exámenes

```
SELECT Leg, AVG(Nota) AS Promedio
FROM Examen
GROUP BY Leg
HAVING COUNT(*) > 2;
```

Leg	Promedio
69	6.00

Primero se agrupa por alumno. Luego HAVING conserva solo los grupos con más de 2 exámenes. El único que cumple es el legajo 69: tiene 3 y su promedio es 6.

10. Cantidad de aprobados y promedio mayor a 5

Siguiendo el ejemplo 39 del apunte, se consideran aprobados los exámenes con **Nota >= 4**:

```
SELECT Leg, COUNT(*) AS Aprobados
FROM Examen
WHERE Nota >= 4
GROUP BY Leg
HAVING AVG(Nota) > 5
ORDER BY Leg;
```

Leg	Aprobados
69	3
109	1
160	1

> **Qué promedio se está calculando:** WHERE elimina primero los desaprobados, así que HAVING evalúa el promedio de las notas aprobadas. El legajo 109 tiene notas 2 y 8: su promedio general es 5, pero acá queda solo el 8 y por eso aparece. Si te pidieran el promedio general, habría que plantear otra consulta.

La diferencia que conviene recordar

Cláusula	Qué filtra	Ejemplo
WHERE	Filas, antes de agrupar.	Nota >= 4
HAVING	Grupos, después de agrupar.	AVG(Nota) > 5

No escribas WHERE COUNT(*) > 2: el conteo del grupo se evalúa en HAVING. Si una condición se refiere a la fecha o a la nota de cada examen, corresponde al WHERE.

Orden de escritura de las cláusulas

```
SELECT columnas y funciones
FROM tablas y juntas
WHERE condición de fila
GROUP BY columnas de agrupación
HAVING condición de grupo
ORDER BY columnas de ordenamiento;
```

Para razonar la consulta: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Este es el orden lógico; el motor puede optimizar la ejecución física.